

(0 Points) Examine the following C code snippet carefully:

```
void process_data(const char* input) {  
    char buffer[16];  
    strncpy(buffer, input, sizeof(buffer));  
    printf("DataProcessed:%s\n", buffer);  
}
```

If the function is called with a string `input` of length 20 (e.g., "ThisIsASecretMessage"), which of the following statements is **TRUE**?

- A. The code is completely safe because `strncpy` prevents writing beyond the 16th byte of `buffer`.
- B. A Stack Buffer Overflow occurs because `strncpy` will force a null-terminator at `buffer[16]`.
- C. A Buffer Over-read may occur during `printf` because `buffer` will not be null-terminated.
- D. The code will trigger a segmentation fault during `strncpy` because the input is stored in read-only memory.

(0 Points) Alice and Bob agree to use the textbook Diffie-Hellman key exchange protocol with the public parameters $p = 31$ (modulus) and $g = 3$ (generator).

- (a) Alice chooses her private key $a = 12$. Calculate her public key $A = g^a \pmod{p}$.
- (b) Bob chooses his private key $b = 7$. Calculate his public key $B = g^b \pmod{p}$.
- (c) Alice receives B and computes the shared secret $K = B^a \pmod{p}$. Calculate K .

(0 Points) Consider the multiplicative group $G = (\mathbb{Z}_{55}^*, \cdot)$.

- (a) True or False: The group G is a cyclic group.
- (b) What is the order of the group $|G|$?
- (c) True or False: The element $g = 2$ is a generator of G .
- (d) Calculate $2^{126} \pmod{55}$. Please express all answers in this assignment involving modular arithmetic (e.g., $\pmod{n}$) as the smallest integer in the range $[0, n - 1]$.

(0 Points) **True or False:** Let $p = 23$. $g = 5$ is a generator of $\mathbb{Z}_{23}^*$. Alice and Bob perform a Diffie-Hellman exchange. An eavesdropper intercepts $A = g^a = 8$ and $B = g^b = 10$. They are also given a challenge value Z , which is either $Z_0 = g^{ab} \pmod{23}$ or $Z_1 \xleftarrow{\$} \mathbb{Z}_{23}^*$. If the eavesdropper receives $Z = 7$, they can determine with 100% certainty that Z is a random element (Z_1) by checking the Legendre symbol $\left(\frac{x}{p}\right) \equiv x^{(p-1)/2} \pmod{p}$.

(0 Points) **True or False:** Consider a standard RSA-CRT implementation where the decryption (or signing) process is optimized using the Chinese Remainder Theorem to compute $M^d \pmod{n}$. The RSA-CRT construction is IND-CPA secure.

(0 Points) Let $p = 197$ and consider the multiplicative group $G = (\mathbb{Z}_{197}^*, \cdot)$. We want to find the smallest positive integer x such that:

$$2^x \equiv 14 \pmod{197}$$

- (a) True or False: The order of the element $g = 2$ in $\mathbb{Z}_{197}^*$ is 196.
- (b) Calculate $2^{98} \pmod{197}$.
- (c) Calculate $2^{50} \pmod{197}$.
- (d) What is the value of x ?

(0 Points) Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ be a secure PRG where $\ell(n) > 2n$. Bob attempts to construct a new PRG $G_{new} : \{0, 1\}^n \rightarrow \{0, 1\}^{2\ell(n)}$. The output is defined as:

$$G_{new}(s) := G(s) \parallel (G(s \oplus 1^n) \oplus G(s)).$$

Is G_{new} a secure PRG?

- **A. Yes.** The security follows from the security of G via a hybrid argument. If an adversary could distinguish $G_{new}(s)$ from a random string $r_1 \parallel r_2$, they would necessarily be able to distinguish $G(s)$ from a random string r_1 , which contradicts the assumption that G is secure.
- **B. Yes.** Since s and $s \oplus 1^n$ are distinct seeds, the outputs $G(s)$ and $G(s \oplus 1^n)$ are computationally independent and pseudorandom. XORing them with each other further "shuffles" the entropy, maintaining security.
- **C. No.** An adversary $\mathcal{D}$ can distinguish the output by receiving $y_1 \parallel y_2$, then computing $y_1 \oplus y_2$.
- **D. No.** The output length $2\ell(n)$ is larger than the stretch allowed for a secure PRG derived from a seed of length n , violating the expansion factor limits of the standard model.

(0 Points) A developer is implementing a login feature and writes the following pseudocode to prevent SQL injection by manually escaping single quotes:

```
$username = $_POST['username'];  
$password = $_POST['password'];  
  
$safe_username = str_replace("'", "''", $username);  
  
$sql = "SELECT * FROM users  
      WHERE username = '$safe_username'  
      AND password = $password";  
  
$result = $db->execute($sql);
```

Which of the following statements best describes the security of this code?

- A. The code is secure because all single quotes in the username are escaped, preventing an attacker from breaking out of the string literal.
- B. The code is vulnerable to SQL injection because the `$password` variable is not enclosed in single quotes and is not sanitized.
- C. The code is vulnerable to "Second-Order SQL Injection" because the escaped string is stored in the database.
- D. The code is secure against SQL injection but vulnerable to Cross-Site Scripting (XSS) due to the use of `str_replace`.

(0 Points) Alice uses the ElGamal encryption scheme with the public parameters $p = 197$ (modulus) and $g = 2$ (generator). Her public key is $h = 14 \pmod{197}$.

- (a) Calculate Alice's private key x , which is the smallest positive integer satisfying $g^x \equiv h \pmod{197}$.
- (b) Bob wants to send a message $m = 50$ to Alice. He chooses a random key $k = 98$. Calculate the ciphertext (c_1, c_2) , where:

$$c_1 = g^k \pmod{197}, \quad c_2 = m \cdot h^k \pmod{197}$$

- (c) The decryption process involves calculating the shared secret $s = c_1^x \pmod{197}$. Calculate s .

(0 Points) An inexperienced engineer decides to use $e = 2$ as the public exponent for a textbook RSA system to speed up encryption, with the public modulus $n = 437$.

- (a) Explore why this system is not a valid RSA cryptosystem by calculating $\gcd(e, \phi(n))$. (Note: $n = 19 \times 23$).
- (b) Given a ciphertext $C = 139$, the attacker wants to recover the plaintext M . Calculate the smallest positive one among the possible plaintexts. (Hint: For $x^2 \equiv a \pmod{p}$, if $p \equiv 3 \pmod{4}$, then $x \equiv \pm a^{(p+1)/4} \pmod{p}$).
- (c) From the second question, we know incorrectly set $e = 2$ will break the 1-to-1 mapping between the plaintext and the ciphertext. Suppose the engineer restarts the scheme with $e = 2, n' = 323$. An attacker obtains two possible plaintexts $M_1 = 4$ and $M_2 = 72$ when decrypting a ciphertext. Help the attacker to recover p, q such that $p * q = n' = 323$ and $p < q$.